# PSVO Zadanie 1 - dokuementácua

Tadeáš Kapina, Radoslav Janíček

March 16, 2026

## 1 Kalibrácia Kamery

Kalibrácia kamery prebehla podľa oficiálneho návodu OpenCV. Nasnímali sme 22 snímok šachovnice, na ktorých prebehla kalibrácia. Snímky sme nasnímali pred samotným kalibračným procesom a uložili ich do priečinka - snímky sme ukladali iba keď na nich OpenCV dokázal rozoznať šachovnicu.

Následne sme použili funkciu `cv.findChessboardCorners()` na nájdenie rohov a následne `cv.cornerSubPix()`, ktorá slúži na spresnenie detektie.

Nakoniec sme použili funkciu `cv2.calibrateCamera()` na získanie matice vnútorných parametrov kamery a disorčných koeficientov.

### 1.1 Parametre kamery

**Matica vnútorných parametrov kamery**

$$K = \begin{pmatrix} 600.352 & 0 & 182.412 \\ 0 & 722.178 & 192.004 \\ 0 & 0 & 1 \end{pmatrix}$$

**Vnútorné parametre kamery**

- $f_x = 600.352$
- $f_y = 722.178$
- $c_x = 182.412$
- $c_y = 192.004$

**Disorčné koeficienty**

$$D = \begin{pmatrix} -0.399 & -0.417 & 0.001 & 0.003 & 3.403 \end{pmatrix}$$

### 1.2 Demonštrácia na reálnom obraze

Figure 1: Vľavo: pôvodný obraz, vpravo: obraz po odstránení disorzie (`cv2.undistort`)

# 2 Detekcia geometrických tvarov

## 2.1 Použité metódy

Na detekciu základných geometrických tvarov v obraze boli využité metódy a funkcie z knižnice OpenCV:

- **Detekcia kružníc:** Využíva sa Houghova transformácia na hľadanie kruhov (`cv2.HoughCircles`). Metóda sa aplikuje na obraz, ktorý bol najprv prevedený do odtieňov sivej a vyhladený mediánovým filtrom pre zníženie šumu.
- **Detekcia polygónov (štvorce, obdĺžniky, trojuholníky):**
  - Na nájdenie hrán v obraze je použitý Cannyho detektor hrán (`cv2.Canny`).
  - Následne program hľadá spojené oblasti (kontúry) pomocou funkcie `cv2.findContours`.
  - Nájdené kontúry sú aproximované pomocou `cv2.approxPolyDP` na zistenie počtu vrcholov (napríklad 3 vrcholy pre trojuholník, 4 pre štvorec alebo obdĺžnik).
- **Určenie ťažiska:** Stred (ťažisko) každého detegovaného tvaru sa vypočítava z obrazových momentov pomocou funkcie `cv2.moments`.

## 2.2 Postup riešenia

Postup spracovania obrazu a detekcie tvarov funguje nasledovne:

1. Program v nekonečnej slučke načítava obraz z kamery.
2. Snímka sa konvertuje do grayscale formátu a filtruje sa, aby sa pripravila na detekciu.
3. Pre zabezpečenie robustnosti voči rôznym svetelným podmienkam a scénam program obsahuje grafické rozhranie s posuvníkmi (Trackbars). Tieto umožňujú používateľovi v reálnom čase ladiť prahy pre Cannyho detektor, parametre Houghovej transformácie, úroveň aproximácie a minimálnu plochu objektov.
4. Ak program deteguje tvar, graficky ho v reálnom obraze označí (vykreslí obrys zelenou farbou).
5. Do stredu tvaru sa vykreslí červený bod, ktorý reprezentuje jeho ťažisko.
6. Ku každému rozpoznanému tvaru program pridá textovú menovku s jeho názvom v reálnom čase (napr. `Tri`, `Square`, `Rect`, `Circle`).

### 2.3 Demonštrácia na reálnom obraze

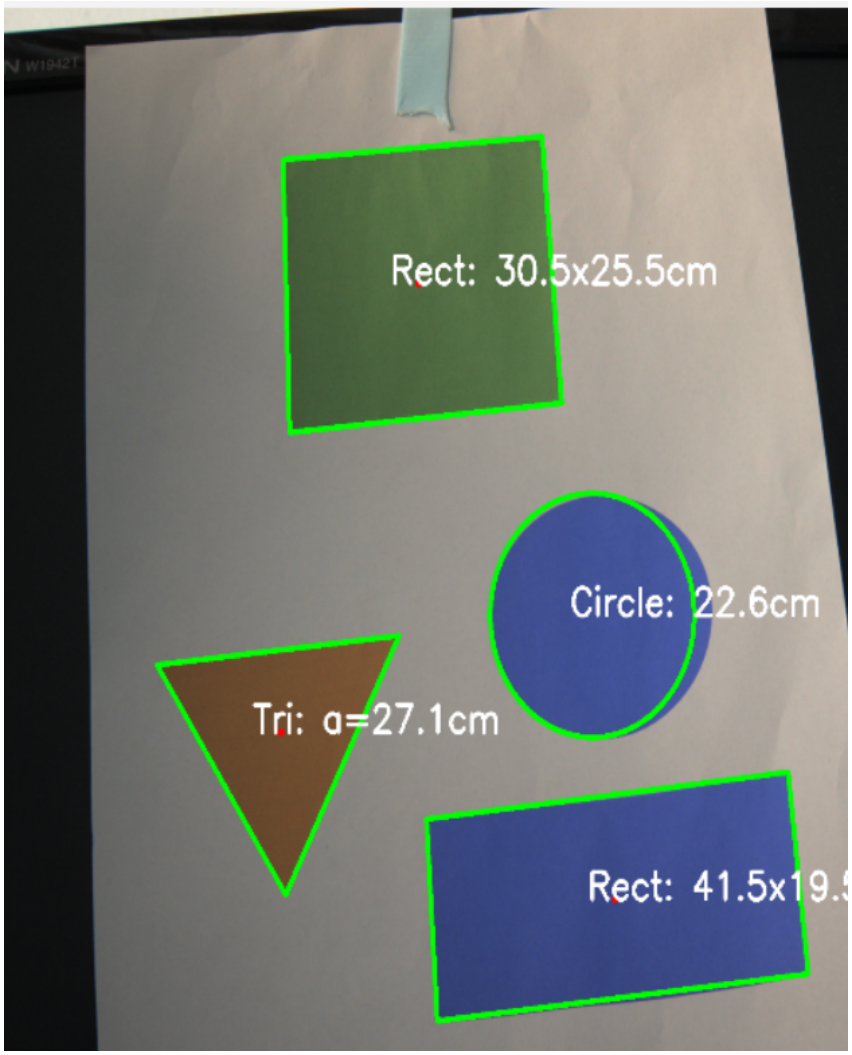


Figure 2: Demonštrácia detekcie geometrických tvarov.

# 3 Farebný filter

Pre zmenu jednej farby na iný sme najskôr zmenili farebný priestor z pôvodného RGB z kamery na HSV pomocou funkcie `cv2.cvtColor`. Následne sme použili funkciu `cv2.inRange` na vytvorenie masky, ktorá vyberá pixely v zadanom rozsahu farieb v HSV priestore. Nakoniec sme všetky pixely, ktoré patrili do masky, zmenili na požadovanú farbu.

## 3.1 Demonštrácia na reálnom obraze

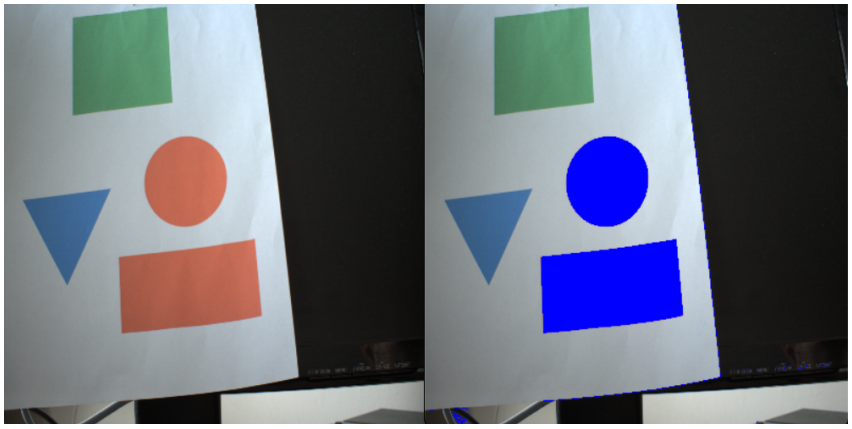

Figure 3: Demonštrácia farebného filtra. Vľavo: pôvodný obraz, vpravo: obraz po aplikovaní farebného filtra.